

UNIVERSIDAD TECNOLÓGICA DEL PERÚ - UTP

LABORATORIO N° 2

ALGORITMOS Y DISEÑO DE ALGORITMOS

CURSO : Análisis y Diseño de Algoritmos

DOCENTE : José Antonio Rojas Guillen

INTEGRANTES :

- Campos Rutti, Kenny Leonardo
- Velasco Socualaya, Stephano
- Salas Pocomucha, Enzo Fabricio
- Cordova Huaynalaya, Angel Jeremy

HUANCAYO - PERÚ

2026

1. ¿Cuál fue el primer intercambio realizado?

El primer intercambio fue entre los valores 12 y 15, en las posiciones 0 y 4, quedando la lista: [15, 5, 9, 3, 12, 1].

2. ¿Cuántos intercambios hubo en total?

En total hubo 12 cambios.

3. ¿En qué pasada la lista quedó completamente ordenada?

La lista quedó completamente ordenada después de la última pasada, cuando $i = 5$. Si contamos las pasadas desde 1 sería en la sexta pasada.

Espacio para evidencias (captura de pantalla del Colab o salida):

```

▶ lista = [12,5,9,3,15,1]
print("arreglo original",lista)
aux = 0
contador = 0
for i in range(len(lista)):
    for j in range(len(lista)):
        if lista[i] < lista[j]:
            aux = lista[i]
            contador+=1
            lista[i] = lista[j]

            lista[j] = aux
            print(f"Ordenamiento: {lista}")
print("arreglo final",lista)
print("Numero de intercambios = ", contador)

... arreglo original [12, 5, 9, 3, 15, 1]
Ordenamiento: [15, 5, 9, 3, 12, 1]
Ordenamiento: [5, 15, 9, 3, 12, 1]
Ordenamiento: [5, 9, 15, 3, 12, 1]
Ordenamiento: [3, 9, 15, 5, 12, 1]
Ordenamiento: [3, 5, 15, 9, 12, 1]
Ordenamiento: [3, 5, 9, 15, 12, 1]
Ordenamiento: [3, 5, 9, 12, 15, 1]
Ordenamiento: [1, 5, 9, 12, 15, 3]
Ordenamiento: [1, 3, 9, 12, 15, 5]
Ordenamiento: [1, 3, 5, 12, 15, 9]
Ordenamiento: [1, 3, 5, 9, 15, 12]
Ordenamiento: [1, 3, 5, 9, 12, 15]
arreglo final [1, 3, 5, 9, 12, 15]
Numero de intercambios = 12

```

4. ¿Qué número fue seleccionado primero?

El primer número seleccionado fue el 2 (que estaba en la posición 3). Se intercambió con el 20 de la posición 0.

5. ¿Qué número fue seleccionado en la segunda pasada?

En la segunda pasada se seleccionó el 6 (estaba en la posición 5) y se intercambió con el 8 de la posición 1.

6. ¿Cuántas pasadas se realizaron?

Se realizaron 5 pasadas (para una lista de 6 elementos, $n-1 = 5$).

Espacio para evidencias (captura de pantalla del Colab o salida):

```

▶ lista = [20, 8, 14, 2, 11, 6]
print("Arreglo original:", lista)
contador = 0
for i in range(len(lista)-1):
    min_val = min(lista[i:])
    pos = lista.index(min_val, i)
    if pos != i:
        lista[i], lista[pos] = lista[pos], lista[i]
        contador += 1
        print(f"Pasada {i+1}: Se seleccionó {min_val} en posición {pos}, intercambiado con {lista[i]}")
    else:
        print(f"Pasada {i+1}: El menor ya está en su lugar ({min_val})")
        print(f"Lista después de pasada {i+1}: {lista}")
print("Arreglo final:", lista)
print("Número de intercambios =", contador)

... Arreglo original: [20, 8, 14, 2, 11, 6]
Pasada 1: Se seleccionó 2 en posición 3, intercambiado con 20
Lista después de pasada 1: [2, 8, 14, 20, 11, 6]
Pasada 2: Se seleccionó 6 en posición 5, intercambiado con 8
Lista después de pasada 2: [2, 6, 14, 20, 11, 8]
Pasada 3: Se seleccionó 8 en posición 5, intercambiado con 14
Lista después de pasada 3: [2, 6, 8, 20, 11, 14]
Pasada 4: Se seleccionó 11 en posición 4, intercambiado con 20
Lista después de pasada 4: [2, 6, 8, 11, 20, 14]
Pasada 5: Se seleccionó 14 en posición 5, intercambiado con 20
Lista después de pasada 5: [2, 6, 8, 11, 14, 20]
Arreglo final: [2, 6, 8, 11, 14, 20]
Número de intercambios = 5

```

7. ¿Qué elemento se insertó en el segundo paso?

El segundo paso (índice $i=2$) insertó el número 10 (que estaba en la posición 2).

8. ¿Cuál fue el elemento que más se desplazó?

El elemento que sufrió más desplazamientos fue el 18. Fue movido hacia la derecha en cada uno de los cinco pasos (se desplazó 5 veces en total), mientras que otros elementos como el 10 o el 7 se movieron menos veces.

9. ¿Qué parte de la lista ya estaba ordenada después de la tercera pasada?

Después de la tercera pasada (cuando se insertó el 4), la lista quedó como [4, 7, 10, 18, 13, 2]. En ese momento, los primeros 4 elementos (índices 0 a 3) ya están ordenados de forma creciente: [4, 7, 10, 18]. El resto (13, 2) aún no está ordenado.

Espacio para evidencias (captura de pantalla del Colab o salida):

```

▶ lista = [18, 7, 10, 4, 13, 2]
print("Arreglo original:", lista)
aux = 0
movimientos = 0
for i in range(1, len(lista)):
    key = lista[i]
    j = i - 1
    print(f"Paso {i}: Insertando {key} (estaba en posición {i})")

    while j >= 0 and lista[j] > key:
        lista[j + 1] = lista[j]
        j -= 1
        movimientos += 1
    lista[j + 1] = key
    print(f"Lista después de inserción: {lista}")

print("Arreglo final:", lista)
print("Número de desplazamientos:", movimientos)

... Arreglo original: [18, 7, 10, 4, 13, 2]
Paso 1: Insertando 7 (estaba en posición 1)
Lista después de inserción: [7, 18, 10, 4, 13, 2]
Paso 2: Insertando 10 (estaba en posición 2)
Lista después de inserción: [7, 10, 18, 4, 13, 2]
Paso 3: Insertando 4 (estaba en posición 3)
Lista después de inserción: [4, 7, 10, 18, 13, 2]
Paso 4: Insertando 13 (estaba en posición 4)
Lista después de inserción: [4, 7, 10, 13, 18, 2]
Paso 5: Insertando 2 (estaba en posición 5)
Lista después de inserción: [2, 4, 7, 10, 13, 18]
Arreglo final: [2, 4, 7, 10, 13, 18]
Número de desplazamientos: 11

```

10. ¿Qué cambio hiciste en la condición de comparación?

Cambié la condición de `lista[i] < lista[j]` usada para orden ascendente a `lista[i] > lista[j]`. De esta forma, cuando el elemento en la posición *i* es mayor que el de la posición *j*, se intercambian, dejando los números más grandes al principio de la lista.

11. ¿Cuál fue el resultado final?

El resultado final es la lista ordenada de forma descendente: [18, 13, 10, 7, 4, 2].

12. ¿Qué algoritmo elegiste y por qué?

Elegí el algoritmo de intercambio porque es el que ya habíamos implementado y solo requiere modificar la condición de comparación para invertir el orden. Así mantengo consistencia con los ejercicios anteriores y el código es fácil de adaptar.

Espacio para evidencias (captura de pantalla del Colab o salida):

```

▶ lista = [9, 17, 4, 12, 6, 1]
print("Arreglo original:", lista)
aux = 0
contador = 0

for i in range(len(lista)):
    for j in range(len(lista)):
        if lista[i] > lista[j]:
            aux = lista[i]
            lista[i] = lista[j]
            lista[j] = aux
            contador += 1
    print(f"Después de pasada {i+1}: {lista}")

print("Arreglo final (descendente):", lista)
print("Número de intercambios:", contador)

... Arreglo original: [9, 17, 4, 12, 6, 1]
Después de pasada 1: [1, 17, 9, 12, 6, 4]
Después de pasada 2: [17, 1, 9, 12, 6, 4]
Después de pasada 3: [17, 9, 1, 12, 6, 4]
Después de pasada 4: [17, 12, 9, 1, 6, 4]
Después de pasada 5: [17, 12, 9, 6, 1, 4]
Después de pasada 6: [17, 12, 9, 6, 4, 1]
Arreglo final (descendente): [17, 12, 9, 6, 4, 1]
Número de intercambios: 8

```

13. ¿Qué algoritmo representa este código?

Es el algoritmo de ordenamiento por selección. En cada iteración del bucle externo (i) se compara el elemento actual con todos los elementos siguientes (j) y se intercambia cuando corresponde, hasta ir seleccionando y colocando el valor correcto en cada posición.

14. ¿Por qué está mal si se desea ordenar de menor a mayor?

Porque la condición original era `if lista[i] < lista[j]`: esa condición intercambia los valores cuando el elemento actual es menor que el siguiente, lo cual va colocando los números más grandes primero. Eso produce un orden descendente no ascendente. Para ordenar de menor a mayor se debe intercambiar cuando el elemento actual sea mayor que el siguiente

15. ¿Qué línea o condición corregiste?

La línea: `if lista[i] < lista[j]`:

se cambió por: `if lista[i] > lista[j]`:

Espacio para evidencias (captura de pantalla del Colab o salida):

```
▶ lista = [10, 3, 7, 1, 5]

for i in range(len(lista)):
    for j in range(i + 1, len(lista)):
        if lista[i] > lista[j]:
            aux = lista[i]
            lista[i] = lista[j]
            lista[j] = aux

print("Lista ordenada:", lista)

... Lista ordenada: [1, 3, 5, 7, 10]
```